TECHNICAL DOCUMENTATION

Safe Route AI

Security threat reporting and route advisory for humanitarian teams operating in North Eastern Kenya.

> **Report → Analyse → Verify → Alert → Protect**

A mobile application that centralises security incident reporting, uses AI to classify reports and detect duplicates, requires human verification before any threat goes live, and delivers location-aware alerts and route advisories to the workers those threats affect.

Platform	**React Native (Expo SDK 57) — Android, iOS, Web**
Backend	**Supabase — PostgreSQL 17 + PostGIS, Auth, Realtime**
Language	**TypeScript (strict)**
Application code	**109 files · ~25,400 lines**
Database	**11 migrations · ~2,000 lines SQL**
Automated tests	**149 assertions across 3 suites**
Status	**Frontend complete · Backend live · AI pending deployment**

Contents

1. **The Problem and the Response**

   Operating context · Why verification is the core

2. **Roles and Separation of Duties**

   Permission matrix · Why admins cannot verify

3. **System Architecture**

   Layers · Data flow · The transport seam

4. **Data Model**

   Tables · Constraints the database enforces

5. **Security Model**

   Row Level Security · Reporter anonymity · Audit trail

6. **The Alert Fan-Out**

   The four rules · Deliberate exclusions

7. **AI Classification**

   Advisory-only design · Duplicate detection · Oversight metrics

8. **Offline Resilience**

   The outbox · Idempotency · Cache policy

9. **Safe Route Advisory**

   PostGIS analysis · Risk scoring

10. **SOS**

    Trigger design · Response console

11. **Design System**

    Dark-first rationale · Validated severity palette

12. **Testing and Verification**

13. **Deployment and Operations**

14. **Known Limitations and Roadmap**

1. The Problem and the Response

Humanitarian organisations in Garissa, Wajir and Mandera counties operate in an environment of armed incidents, road ambushes, banditry and inter-communal conflict. Security information exists, but it moves by phone call, WhatsApp group and word of mouth. Three failures follow from that:

- **Information does not reach the people it concerns.** A road ambush is known to one team and unknown to the convoy that leaves an hour later.
- **Unverified rumour circulates as fact.** A second-hand report can shut down a corridor that was never actually dangerous.
- **Nothing is auditable.** After an incident, there is no record of who knew what, or when.

Safe Route AI addresses these with a single closed loop.

STAGE	WHAT HAPPENS	WHO ACTS
Report	Free-text description, optional photo, GPS captured automatically. Works with no connectivity.	Humanitarian worker
Analyse	AI classifies threat type and severity, suggests an alert radius, and flags likely duplicates.	System (advisory only)
Verify	An officer confirms or overrides every field. Nothing reaches the field without this step.	Security officer
Alert	Fan-out to affected workers by organisation, partnership, proximity and declared region.	System
Protect	Threat map, route risk comparison, one-tap SOS.	Humanitarian worker

> **THE CENTRAL DESIGN DECISION**
>
> **Verification by a human is mandatory and cannot be bypassed.** The AI proposes; a security officer decides. This is enforced at the database level, not merely in the interface — a client that bypassed the app entirely still could not publish an unverified threat. Everything else in the system is arranged around protecting that property.

Operating constraints that shaped the build

CONSTRAINT	CONSEQUENCE FOR THE DESIGN
Connectivity gaps	Reports and SOS are recorded on the device before any network call, and send themselves later. Nothing blocks on the network.
Retaliation risk	Reporter identity is stripped and positions blurred before a report reaches a peer — at the data layer, not in a component.
Night and vehicle use	Dark-first interface. A bright screen is both uncomfortable and a way of making yourself visible.
Decisions under stress	Severity is encoded by colour, word and size. SOS requires a deliberate hold, not a tap.
Accountability after incidents	Append-only audit log that no client can write to or suppress.

2. Roles and Separation of Duties

Three roles, with boundaries enforced by Row Level Security in PostgreSQL rather than by hiding controls in the interface.

CAPABILITY	WORKER	OFFICER	ADMIN
File a threat report	YES	YES	NO
See verified threats	ANONYMISED	FULL	FULL
See pending reports	OWN ONLY	YES	READ-ONLY
Verify or dismiss a report	NO	YES	NO
Set the alert radius	NO	YES	NO
Issue a standalone advisory	NO	YES	NO
Trigger an SOS	YES	YES	NO
Respond to an SOS	NO	YES	NO
Invite and manage people	NO	NO	YES
Manage partnerships	NO	NO	YES
Read the audit log	NO	YES	YES

WHY AN ADMIN CANNOT VERIFY A REPORT

An admin creates the officer accounts. If the same person could also exercise an officer's authority, a single account could both grant the power to publish threats and use it — and the audit trail would no longer show an independent decision. Splitting the two means every verified threat carries the name of someone who was appointed to that role by a different person.

The same reasoning excludes officers from creating invites, and excludes everyone from writing to the audit log.

Cross-organisation sharing

Organisations may declare cross-share partnerships. Partnerships are **reciprocal by construction** — enforced by a database trigger that mirrors the relationship — because a one-way arrangement would let an organisation consume another's threat intelligence without contributing any.

DATA	SHARED WITH PARTNERS?
Verified threat reports	YES
Safety advisories (opt-in per advisory)	YES
Pending, unverified reports	NO
Staff directory	NO
Live SOS locations	NEVER

A live worker position is the most sensitive datum in the system. Partner organisations share threats; they do not share people.

3. System Architecture

```
src/
  api/                  the backend seam
    client.ts           mock transport: latency, offline, rate limits, role guards
    db.ts               in-memory store, realtime channels, audit
    seed.ts             deterministic demo dataset
    cache.ts            stale-while-revalidate, disk persistence, tag invalidation
    rateLimit.ts        token buckets, persisted across restarts
    routing.ts          route construction and risk scoring (shared)
    realtime.ts         one subscribe() over both transports
    endpoints/          auth · reports · alerts · sos · org · analytics · routes
    supabase/           client · mappers · call · one module per endpoint
  ai/classifier.ts      classification and duplicate detection
  lib/                  geo · offlineQueue · validation · logger · push · network
  state/                auth · connectivity+outbox · notifications · toast
  components/ui/        design system primitives
  components/domain/    threat cards, AI panel, verify sheet, maps, charts
  app/                  32 routes, one group per role

supabase/
  migrations/           11 versioned migrations
  seed.sql              demo data, applied through the real invite flow
```

The transport seam

The application supports two interchangeable backends behind one interface. Screens import an endpoint and never learn which answered:

```
export const reportsApi = API_MODE === 'supabase'
  ? supabaseReportsApi
  : mockReportsApi;
```

The mock is a **first-class implementation**, not a migration stub. It runs entirely on-device with a deterministic dataset, needs no network and no project, and is switched on with a single environment variable. It exists so the application can be demonstrated, reviewed and developed without infrastructure — and so the offline paths can be exercised deliberately.

Data flow: filing through to alerting

```
Worker files a report
  → client mints a UUID; zod validates the draft
  → OFFLINE?  outbox (persisted) → backoff retries → same id, no duplicate
  → ONLINE?   INSERT ... status 'pending'   [RLS forbids any other status]
              → classifier writes ai_suggestions   [service role only]
              → Realtime pushes it to the officer queue

Officer verifies
  → RLS: officer role required, own or partner organisation
  → officer-confirmed columns written; ai_suggestions left untouched
  → trigger stamps verified_by / verified_at from the JWT, not the payload
  → trigger appends an audit entry naming every overridden field
  → fan_out_alert() applies the four rules in one SQL statement

Worker receives
  → Realtime toast + OS notification + persistent feed row
  → the feed row carries the reasons it matched
```

Cross-platform mapping

`ThreatMap` has two implementations behind one props contract: `react-native-maps` on device, and an SVG schematic on web, where that library has no build. The web version draws real road corridors, real place names and an equirectangular projection — every coordinate, radius and route on it is accurate; it simply has no tiles. Screens import `ThreatMap` and never learn which they received.

4. Data Model

TABLE	PURPOSE AND NOTABLE COLUMNS
`organizations`	Name, acronym, operating regions, reciprocal `cross_share_partners`, per-org policy settings.
`profiles`	One row per auth user. Role, organisation, `operating_regions`, `last_known_location` (geography), notification preferences.
`threat_reports`	The core entity. Reporter-supplied fields, officer-confirmed fields (nullable until verified), and `ai_suggestions` (jsonb, immutable once written).
`safety_alerts`	Threat alerts and standalone advisories. Location, radius, region scope, expiry.
`notifications`	The persistent feed. Carries `match_reasons` so a recipient can see why it reached them.
`sos_events`	Status, live position, position `track` and `timeline` as jsonb.
`invites`	Invite-gated signup. The code binds an account to an organisation and a role.
`audit_log`	Append-only. Actor, action, before/after values.
`regions`	Sub-county reference geography, needed server-side by the fan-out.

Invariants the database refuses to break

These are constraints and triggers, not application conventions. They hold regardless of which client is talking to the database.

RULE	MECHANISM
A verified report must carry threat type, severity and alert radius	`CHECK` constraint
A dismissed report must carry a reason	`CHECK` constraint
A resolved report can never be reopened	Trigger
AI suggestions are written once, by the server, and never edited	Trigger + RLS
Verification is attributed to the acting JWT, not the payload	Trigger
Nobody may change their own role, organisation or status	Trigger
An organisation must always retain one active admin	Trigger
At most one live SOS per worker	Partial unique index
Partnerships are always reciprocal	Trigger
Only `read_at` may change on a notification	Trigger

Geography

Positions are stored as PostGIS `geography(Point, 4326)` with GiST indexes, which is what makes proximity queries fast. Because PostgREST serialises geography as WKB hex, generated `lat` / `lng` columns expose plain numbers to clients. They are `STORED` and recomputed by Postgres on every write, so they cannot drift from the geometry they derive from.

5. Security Model

9	22	19	12
TABLES	RLS POLICIES	DB FUNCTIONS	TRIGGERS

Why Row Level Security carries the weight

The application ships with a public API key embedded in the bundle. Anyone with the installed app can extract it. That is by design and is safe only because **every table denies by default** and opens only what a given authenticated role needs. The client-side role checks are a courtesy that produces better error messages; the policies are the actual guarantee.

Policy helper functions are `SECURITY DEFINER` with a pinned `search_path`. Definer is required because a policy on `profiles` that itself queries `profiles` would recurse infinitely; the pinned search path prevents a caller shadowing `public` with their own tables.

Reporter anonymity

Protecting reporters from retaliation is a safety requirement, not a privacy preference. RLS controls which *rows* are visible but cannot blank a column, so peer-facing access goes through a dedicated view:

- Reporter identity replaced with `null`
- Photograph withheld
- Position offset by 50–300 m — deterministic, so the pin does not wander between loads; far below any alert radius, so the warning stays accurate
- Your own reports are always returned in full

Organisations operating in especially sensitive contexts can additionally hide reporter identity from their own officers.

Authentication

CONTROL	BEHAVIOUR
Account creation	Invite-gated. A database trigger validates the code inside the signup transaction — an invalid code leaves no orphan account.
Password reset	Six-digit code by email, single-use, time-limited, attempt-capped.
Enumeration resistance	Reset requests return an identical response whether or not the address is registered.
Session storage	Platform keychain / keystore on device.
Password change	Revokes all other sessions — the control used when a device is lost.
Deactivation	Immediate; revokes sessions.

Audit trail

The audit log has **no insert, update or delete policy**. Entries are appended exclusively by `SECURITY DEFINER` triggers, so no client can forge or suppress one. Append-only is the property that makes it evidence rather than a changelog.

6. The Alert Fan-Out

A worker is alerted when **any** of four conditions holds. All four are evaluated in a single SQL statement using PostGIS.

#	CONDITION	RATIONALE
1	Same organisation as the report	Colleagues share an operating picture.
2	A cross-share partner of the reporting organisation	Threats do not respect agency boundaries.
3	Last known position inside the alert radius	Proximity is the most urgent case.
4	Threat falls inside a declared operating region	Covers a worker who travels there but is elsewhere today.

Each notification carries the reasons it matched, and the interface displays them. An alert that explains itself — *"you are inside the 12 km radius"* — is actionable in a way that an unexplained one is not.

Deliberate exclusions

- **The reporter is never alerted about their own report.** They already know; alerting them is noise, not safety.
- **Admins are excluded by default.** They manage the organisation and are not in the field. Paging them on every incident trains everyone to ignore alerts.
- **Quiet hours are respected — except for critical.**
- **"Only critical" does not suppress proximity.** Anything inside the radius reaches the user regardless of preference, because that is about their own safety rather than their inbox.

> **PREVIEW BEFORE SENDING**
>
> Before an officer commits, the interface shows exactly how many people will be notified, computed by a function that applies the same four rules as the fan-out itself. Widening a radius from 6 km to 20 km visibly changes the count from four people to thirty-one. The number shown is the number that will actually be reached — not an estimate that can drift.

7. AI Classification

On submission, a report is analysed for threat type, severity, priority, a suggested alert radius, and likely duplication. The output is written to `ai_suggestions` and is **never** written to the officer-confirmed columns.

Advisory by construction

PROPERTY	HOW IT IS GUARANTEED
AI cannot publish a threat	RLS forbids a non-officer changing status; the insert policy forces `pending`.
AI output cannot be edited	A trigger rejects any client write to `ai_suggestions`, including the first.
Proposal and decision stay separable	Stored in different columns; the audit entry records which fields were overridden.
Classification never blocks a report	Runs after the row is committed. A failed classification leaves the report fully reviewable.
Low confidence is stated plainly	A "needs more detail" flag is surfaced rather than smoothed over.

Duplicate detection

Two stages, mirroring how the production function is intended to run:

1. **Geo and time pre-filter** in SQL — reports within roughly 5 km and 12 hours, using the spatial index.
2. **Semantic comparison** over that shortlist, scoring lexical overlap, proximity and recency with smooth decay. Two reports of the same ambush are rarely pinned to the same metre.

Only a strong, corroborated match is flagged, and a flag is a *suggestion*: the officer sees both reports side by side and decides. Confirming a duplicate closes the second report without issuing a second alert.

Oversight metrics

The application reports an **AI agreement rate** — how often officers accepted a suggestion unchanged — and flags rates above 92% as a concern. Too low means the model is noise. Suspiciously high suggests rubber-stamping, which would make the human verification step decorative. The metric is shown to officers as a mirror, not only to management.

8. Offline Resilience

Coverage gaps are the operating norm, not an edge case. Several architectural decisions exist only because of this.

MECHANISM	BEHAVIOUR
Client-minted identifiers	The device generates a UUID before any network call. It doubles as the idempotency key, so a submission retried after a timeout cannot become a second report.
Priority ordering	SOS always jumps ahead of reports and sends the instant signal returns, surviving app restarts.
Capped backoff with jitter	Per item, so one poisoned entry cannot block the queue. Jitter prevents a convoy regaining signal simultaneously from stampeding the server.
Non-retryable failures park	A malformed payload or permission error cannot be fixed by waiting. It is marked failed and surfaced in the Outbox rather than retried indefinitely.
Honest confirmation	The interface says "saved on this device" — never "sent" — until the server has it.

> **WHY CACHING IS A SAFETY FEATURE HERE**
>
> A worker who opens the threat map in a dead zone must still see the threats verified while they had signal. **A blank map reads as "all clear" — the most dangerous thing this application could say.** Caches therefore serve stale data with a visible "last updated" marker rather than showing an empty state or a spinner.

Named cache policies balance freshness against offline usefulness: threat data is briefly fresh but retained for a week; the officer queue is near-live; analytics tolerate being a minute old. A development switch simulates loss of signal so every queue, cache and retry path can be exercised without leaving the building.

9. Safe Route Advisory

A worker enters origin and destination and receives two or three candidate routes, each scored by the verified incidents near it.

Only verified reports count. An unverified rumour must not be able to close a road.

Analysis

For each candidate polyline, PostGIS finds every verified threat within a configurable buffer of the line (`ST_DWithin` against the route geography). Each is scored with two independent decays:

- **Recency** — exponential, roughly a fortnight half-life
- **Proximity** — linear across the buffer

A critical ambush 200 m off the road last night therefore weighs far more than the same event 1.9 km away a month ago, and the score says so.

Presentation

Routes are ranked with incident count, worst severity, distance and estimated time. A route carrying a critical incident inside the buffer is **never** recommended, whatever its aggregate score.

WHAT THE ADVISORY DOES NOT KNOW

Unreported incidents, checkpoints established this morning, road and vehicle condition, and escort arrangements are all invisible to it. The interface says so explicitly. The output ranks options and explains why; it never chooses, because the person in the vehicle knows things the application does not.

10. SOS

The design constraint: the person triggering this may have seconds, one hand, and no signal.

DECISION	REASONING
Hold, not tap	Broadcasting a live position cannot be undone quietly. A 1.6-second hold with a progress ring and haptic feedback is deliberate but still usable under stress.
Always reachable	Present on every worker screen. Nothing is permitted to obscure it, including transient messages — which is why notifications appear at the top of the screen.
Queued first	Recorded on the device before any network attempt, and sent the instant signal returns.
One live event per worker	A second trigger escalates the existing event rather than fragmenting the officer's view across two cards.
Independent position pings	Each is separately retryable. A dropped ping never invalidates the event.
Acknowledgement is distinct from resolution	The worker most needs to know a human has seen it. Acknowledgement is one tap and is timed as its own metric.

Officers see live events with the position track, add notes visible to the worker, and resolve with a required note. The worker may stand down their own event. The full timeline is retained for post-incident review.

11. Design System

Dark-first. The application is used at night, in vehicles and in low-light compounds, where a bright screen is both a comfort problem and a personal security problem. Both schemes are fully defined; no colour falls back to a guess.

The severity palette was computed, not chosen

Severity is the fastest signal in the product, so the ramp was validated rather than eyeballed. An initial palette placed medium and high only ΔE 11.2 apart in normal vision — two adjacent severity levels that look alike, in a safety application. The ramp was re-stepped until both schemes passed:

SCHEME	CVD SEPARATION	NORMAL VISION	CONTRAST
Dark	ΔE 12.9 (deutan)	ΔE 16.3	All ≥ 3:1
Light	ΔE 13.9 (protan)	ΔE 15.5	All ≥ 3:1

Severity is additionally encoded by **pin size** on the map and is **always paired with its word**, so the level never rests on colour alone — which matters for colour-blind users and in direct sunlight, where hue separation collapses.

Other conventions

- Minimum 44 px touch targets — field use means gloves, motion and hurry

- Haptics as an accessibility feature: confirmation you can feel without looking
- Empty states distinguish "nothing exists" from "nothing matches your filter"
- Charts follow one axis, never two; colour never carries identity alone
- Accessibility roles, labels and live regions throughout

12. Testing and Verification

82	39	28	149
LOGIC	SECURITY	INTEGRATION	TOTAL

SUITE	COVERS
Logic `npm test`	Geodesic and point-to-path geometry against known distances; classifier behaviour on real report narratives including duplicate detection and the refusal path; dataset integrity and determinism; validation schemas.
Security `npm run test:rls`	Signs in as real users with the public key and asserts every boundary: anonymous access denied; workers cannot see pending reports, reporter identities, invites or the audit log; workers cannot self-promote; **admins cannot verify or issue alerts**; SOS is not cross-shared; unrelated organisations see nothing; nobody can write the audit log.
Integration `npm run test:transport`	Walks the entire loop against the live database — file, verify, fan out, receive — and asserts idempotency, automatic attribution, reporter exclusion, position blurring, PostGIS route analysis and audit generation.

Also enforced: TypeScript strict mode with no errors, zero lint errors including React Compiler's purity and ref rules, and a clean toolchain check.

Defects the test suites caught

DEFECT	CONSEQUENCE HAD IT SHIPPED
AI suggestions writable by a client on first write	An officer could fabricate an AI recommendation on a report that never had one — the audit trail would show the model agreeing with a decision it never made.
Malformed payloads mapped to a retryable error	An SOS that could never send would retry silently and forever, while the worker believed help was coming.
Classifier missed common phrasing	"Armed men stopped two vehicles" classified as *Other* at 28% confidence, so the corridor was never flagged in Safe Route.
Severity colours too close to distinguish	Two adjacent severity levels indistinguishable at a glance.

13. Deployment and Operations

Commands

```
npm install              install dependencies
npm start                development server (a / i / w)
npm run android          native Android build
npm run web              browser

npm run db:seed          apply all migrations, then demo data
npm run db:seed -- --schema   migrations only

npm test                 logic suite
npm run test:rls         security boundaries
npm run test:transport   end-to-end against the live database
npm run test:all         everything, plus typecheck
```

Configuration

Environment values are injected at build time through a dynamic Expo config.

> **CRITICAL**
>
> Variables prefixed `EXPO_PUBLIC_` are **inlined into the application bundle** and are readable by anyone holding the installed app. Only the public API key belongs there. The service role key — which bypasses Row Level Security entirely — and the AI provider key must exist only as server-side function secrets, and must never appear in the repository.

Operational notes

- **Backups.** The audit log is the accountability record; point-in-time recovery should be enabled before real users.
- **Project pausing.** Free-tier projects pause after inactivity. Unacceptable for a safety application — a paid tier is required in production.
- **Migrations replay cleanly.** Verified by applying twice.
- **Email confirmation** is currently disabled for demonstration and should be re-enabled, with a deep link, before real users.
- **Not indexed.** This is an authenticated tool for named field staff; search visibility is a deliberate non-goal.

14. Known Limitations and Roadmap

ITEM	STATUS
AI classification on new reports	The classifier is implemented and its output shape is the contract the interface is built against, but it has not yet been deployed as a server-side function. Existing reports carry suggestions; newly filed reports remain unclassified until deployment. Requires a paid AI provider key.
Photograph upload	Capture and queueing work; images are held as local URIs pending object storage.
Background push	In-app realtime alerts work. Delivery while the application is closed requires device push tokens and a development build.
Directions API	Route geometry is currently synthesised from real road corridors. The risk analysis is genuine and unchanged; only the geometry becomes road-accurate when a routing provider is connected.
Dual-control verification	Enforced server-side for critical reports; the second-officer interface is not yet built.
Generated database types	Row types are currently loose. Generating types would make every query statically checked against the real schema.

Suggested sequence

1. Deploy the classification function — completes the loop for new reports
2. Object storage for photographs
3. Push tokens and a development build for background delivery
4. Enable backups and email confirmation; move off the free tier
5. Connect a routing provider for road-accurate geometry
6. Field pilot with one organisation before widening

> **CLOSING NOTE**
>
> The properties this system is built to protect are that **no threat reaches the field without a named human approving it**, that **no reporter is exposed to the people they reported on**, and that **nothing is silently lost when the network disappears**. Every architectural decision documented above serves one of those three. Where a trade-off was made, it was made in their favour.